

Security Audit

Manifest (Ledger)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Manifest Ledger Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	14
Conclusion	15
Our Methodology	16
Disclaimers	18
About Hashlock	19

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lifted Initiative team partnered with Hashlock to conduct a security audit of their Manifest Ledger chain. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The Lifted Initiative is revolutionizing the Web3 landscape by developing a comprehensive framework that seamlessly integrates Web2 and Web3 functionalities, empowering users to create bespoke layer 1 networks without requiring prior blockchain expertise. By contributing to the Many Protocol, Lifted ensures interoperability across various modules and networks, fostering a cohesive decentralized ecosystem. Their commitment to accessibility and sustainability is evident through initiatives like the Liftoff Program, which supports Web3 builders in refining their products and strategies, also, the Manifest Network, built on the Cosmos SDK, consists of a blockchain tailored for decentralized AI infrastructure access. Initially employing a Proof of Authority (PoA) model it ensures a secure and efficient launch phase, with a trusted validator set managing consensus.

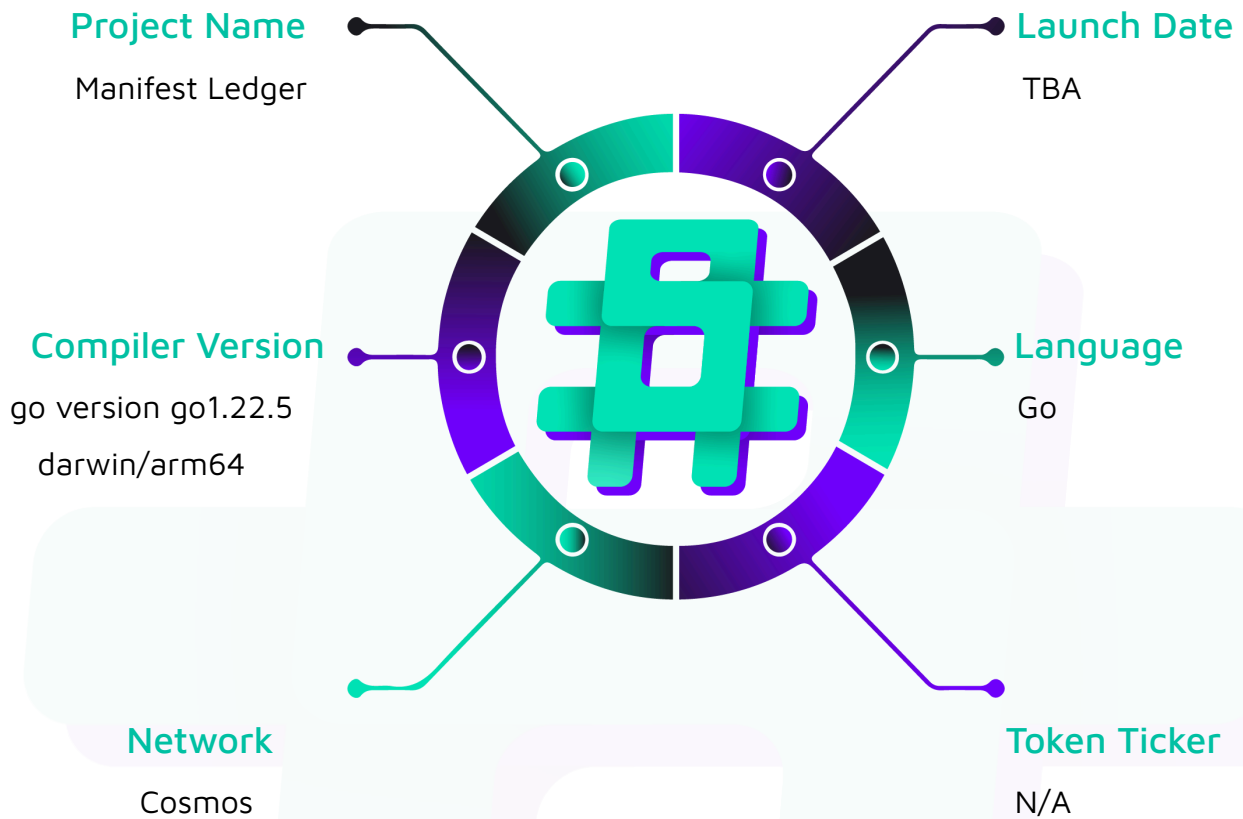
Project Name: Manifest Ledger

Compiler Version: go version go1.22.5 darwin/arm64

Website: <https://liftedinit.org>

Logo:



Visualised Context:

Project Visuals:



What We're Building

We've reimaged the entire tech stack to provide a user-friendly experience. From the framework all the way down to the hardware architecture, our core technology provides a simple yet powerful UX. Oh, and we're also open-source.



The Manifold Framework

A suite of drag-and-drop building block modules that allow you to combine Web2 + Web3 functionality for custom solutions.

[LEARN MORE](#)



The Manifest Network

A decentralized and planet-friendly hardware layer that powers services and allows both deterministic and non-deterministic processing.

[LEARN MORE](#)



The Many Protocol

A communications protocol to exchange messages, data, and assets across sovereign blockchains.

[START BUILDING](#)

[#hashlock.](#)

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the Go code within the Manifest Ledger project, the scope of work included a comprehensive review of the components listed below. We tested the Manifest Ledger to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Manifest Ledger chain
Platform	Cosmos / Go
Audit Date	December, 2024
Repository URL	https://github.com/liftedinit/manifest-ledger
Commit hash	8625d277d31fb699dfb100311861a66fdade9fb2

Description	Manifest Ledger CosmWasm PR (updated version)
Platform	Cosmos / Go
Audit Date	February, 2025
Repository URL	https://github.com/liftedinit/manifest-ledger
Pull request	https://github.com/liftedinit/manifest-ledger/pull/118

Security Rating

After Hashlock's Audit, we found the Manifest Ledger to be **"Hashlocked"**. The codebase all follows simple logic, with correct and detailed ordering. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Manifest Ledger Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low severity vulnerabilities

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Manifest Ledger Functions

Claimed Behaviour	Actual Behaviour
https://github.com/liftedinit/manifest-ledger - Allows the authority to: - Mint coins to specified recipients - Burn coins from any recipients	Module achieves this functionality.

Code Quality

This audit scope involves the codebase of the Manifest Ledger project, as outlined in the Audit Scope section. All components, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Manifest Ledger project components code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this Manifest Ledger infrastructure are based on well-known industry standard open source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

Low

[L-01] `x/manifest/keeper/msg_server.go#BurnHeldBalance` - Message is not validated

Description

In the `BurnHeldBalance` function, the `MsgBurnHeldBalance` request is not validated with the `Validate()` function from `x/manifest/types/messages.go#L111`.

Recommendation

Consider calling `req.Validate()` same like `x/manifest/keeper/msg_server.go#L28`.

Status

Resolved

[L-02] `cmd/manifestd/cmd/root.go#initWasmFlags` - Wasm flags are not added to command line queries

Description

In `cmd/manifestd/cmd/root.go:145`, the `initWasmFlags` function is defined but not used. This means the `wasm.AddModuleInitFlags(startCmd)` will not be called in `cmd/manifestd/cmd/commands.go:128`.

Recommendation

Consider adding `wasm.AddModuleInitFlags(startCmd)` to the `addModuleInitFlags` function in `cmd/manifestd/cmd/commands.go:140`.

Status

Resolved

QA

[Q-01] x/manifest/keeper/quierier.go - Spelling error in filename

Description

quierier.go contains a spelling error in the filename.

Recommendation

Consider updating the filename to querier.go.

Status

Resolved

[Q-02] cmd/manifestd/cmd/root.go - Incorrect srvCfg.MinGasPrices denom

Description

cmd/manifestd/cmd/root.go:133 sets the srvCfg.MinGasPrices to 0stake. In this case, the stake denom is incorrect.

Recommendation

Consider updating the srvCfg.MinGasPrices to use the correct denom or remove it.

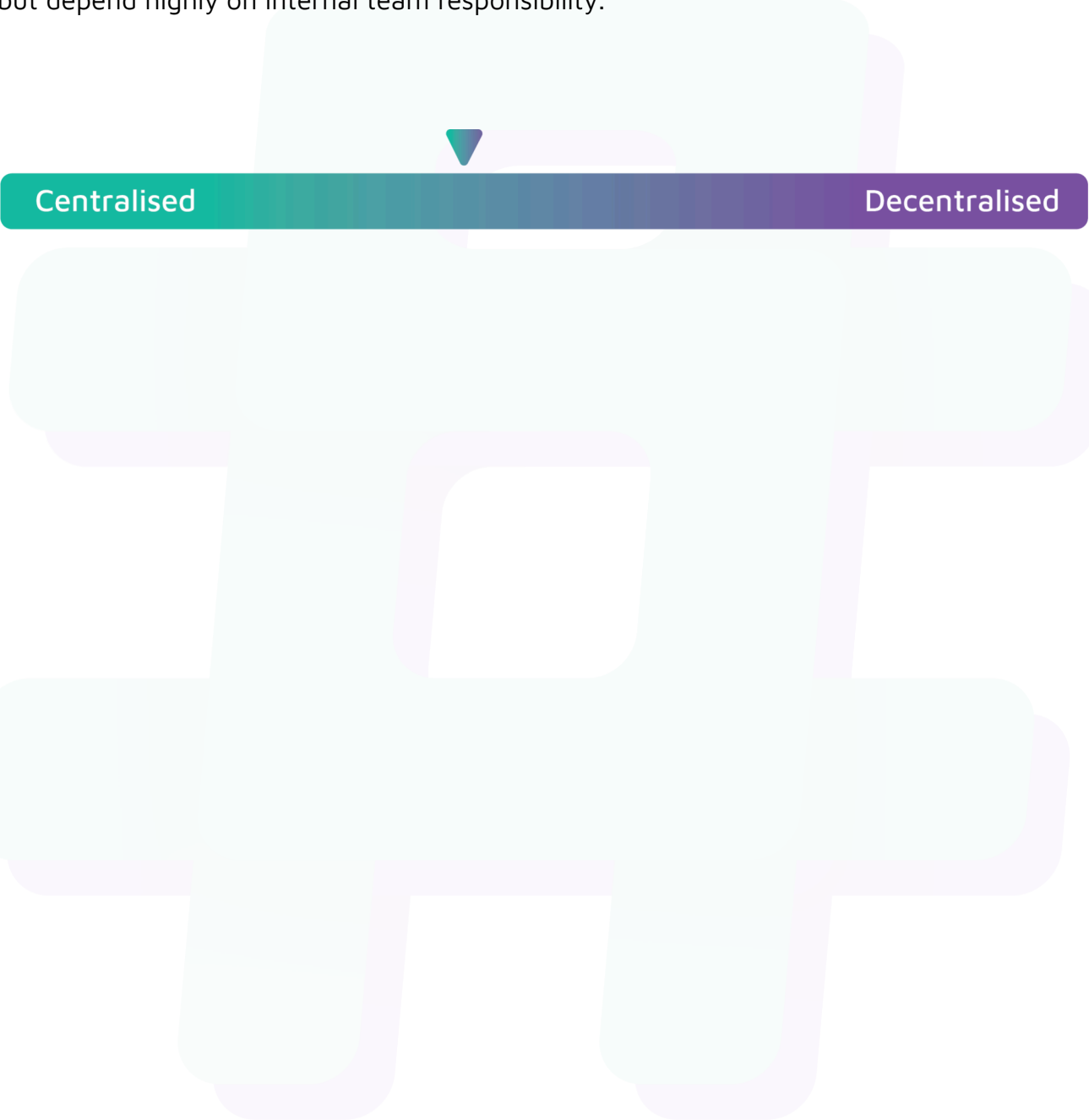
Status

Resolved

Centralisation

The Manifest Ledger project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Manifest Ledger project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the Manifest Ledger under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the Manifest Ledger in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the Ledger's source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this Ledger.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.